

Java Backend Architecture

Interview Series

Chapter 9.8

CDN & Edge Caching

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 9.8 – CDN & Edge Caching

Introduction

Content Delivery Networks (CDNs) are the outermost cache tier — geographically distributed Points of Presence (PoPs) that serve cached content from locations physically close to end users. A request served from a CDN edge node in Frankfurt instead of a data center in Virginia reduces latency from 200ms to 10ms. Leading CDN providers — **CloudFront** (AWS), **Cloudflare**, **Fastly**, and **Akamai** — support sophisticated cache policies, cache tags, edge workers, and origin shield topologies. For Java backends, CDNs are configured primarily via HTTP headers and CDN-specific APIs rather than Java code.

Core Concepts & Definitions

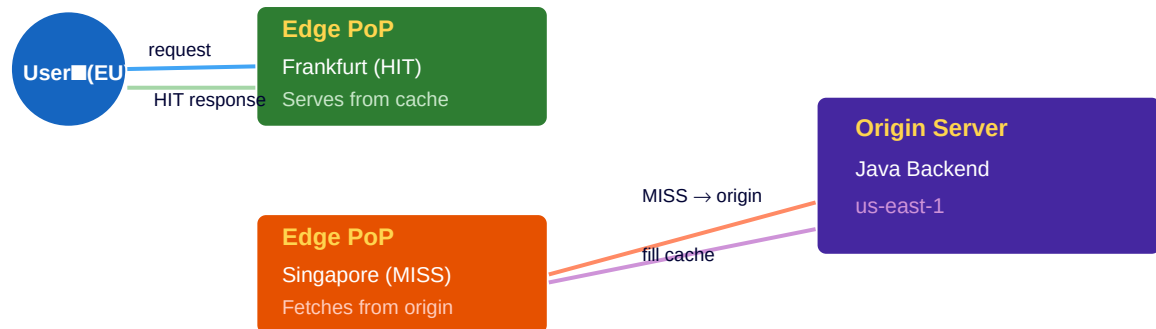
CDN	Content Delivery Network: globally distributed cache servers that serve content from edge nodes closest to users.
Point of Presence (PoP)	A CDN data center in a geographic region; caches and serves content for nearby users.
Edge Node	Individual CDN server at a PoP; stores cached responses and serves them on HIT.
Origin Server	Backend server that CDN fetches from on cache MISS; should set proper Cache-Control headers.
Cache HIT	CDN serves response from its local cache without contacting origin.
Cache MISS	CDN does not have the response; fetches from origin, caches, then serves.
Origin Shield	Single CDN node designated as intermediary between edge nodes and origin; collapses MISS requests.
CloudFront	AWS CDN; integrates with ALB, API Gateway, S3; Lambda@Edge for programmability.
Cloudflare	CDN + DDoS protection + edge workers (Cloudflare Workers in V8); Argo Smart Routing.
Fastly	Programmable CDN with Varnish VCL customization + Compute@Edge (WASM); instant purge API.
Surrogate-Key / Cache-Tag	Header tagging cached responses; allows batch purge of related entries by tag.
Cache Purge / Invalidation	Force CDN to discard cached entries before TTL expires.
Edge Worker	Serverless function running at CDN PoP; can modify requests/responses without origin round-trip.
Lambda@Edge	AWS serverless functions running at CloudFront PoP; 4 trigger points in request/response lifecycle.
ESI (Edge Side Includes)	CDN-level template: assemble responses from multiple cached fragments at edge.

Stale-While-Revalidate

CDN serves stale content while asynchronously refreshing from origin in background.

CDN Request Flow

CDN Request Flow – Global Edge Network



Cache HIT: sub-10ms from edge. Cache MISS: 50-200ms from origin. CDN absorbs 80-95% of traffic.

PoP = Point of Presence. Major CDNs: CloudFront (AWS), Cloudflare, Fastly, Akamai

CDN Cache Invalidation Strategies

CDN Cache Invalidation Strategies

TTL Expiry

Wait for max-age
to expire naturally
Eventual consistency

URL Versioning

Change URL on
content change
Instant, no purge

API Purge

Call CDN API
to invalidate path
Fast, costs \$\$

Surrogate-Key (Cache Tag)

Tag responses;
batch purge by tag
Fastly/Cloudflare

Latency:

TTL: slow | Versioning: instant | Purge: seconds | Tag: seconds

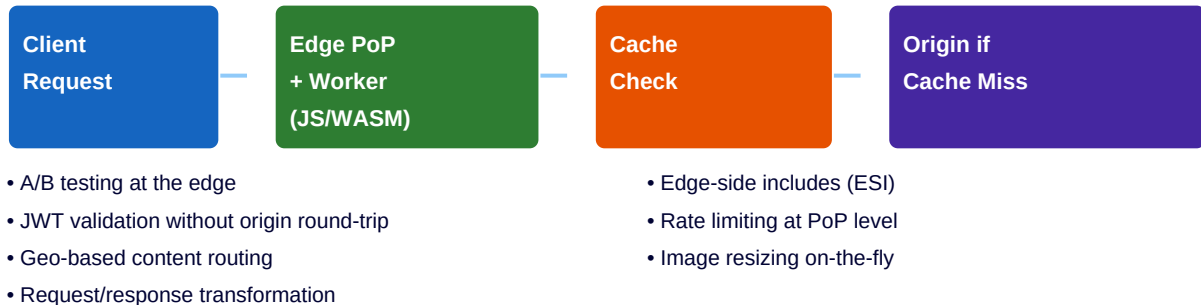
Cost:

TTL: free | Versioning: free | Purge: per-request fee on some CDNs | Tag: requires Enterprise

Best practice: Use URL versioning for static assets + Surrogate-Key for dynamic API content

Edge Workers

Edge Workers / Edge Functions (Cloudflare Workers, Lambda@Edge)



Edge workers reduce origin load and add intelligence at the network edge (<5ms latency)

Setting CDN-Friendly Headers in Spring Boot

```
// Public API responses – CDN cacheable for 5 minutes, browser 1 minute
@GetMapping("/catalog/products") public ResponseEntity<List<Product>> getCatalog() {
    List<Product> products = productService.getAllActive(); return ResponseEntity.ok()
        .cacheControl(CacheControl .maxAge(Duration.ofMinutes(1)) // browser
        .sMaxAge(Duration.ofMinutes(5)) // CDN / shared .staleWhileRevalidate(Duration.ofSeconds(30))
        .cachePublic()) .body(products); } // Versioned API – long CDN cache
@GetMapping("/static/config/v{version}") public ResponseEntity<AppConfig>
getConfig(@PathVariable String version) { AppConfig config =
    configService.getByVersion(version); return ResponseEntity.ok() .cacheControl(CacheControl
        .maxAge(Duration.ofDays(365)) .immutable() .cachePublic()) .body(config); }
```

Surrogate-Key / Cache-Tag (Fastly / Cloudflare)

```
// Tag responses with Surrogate-Key for batch purge
@GetMapping("/products/{id}") public
ResponseEntity<Product> getProduct(@PathVariable Long id) { Product p =
    productService.findById(id); return ResponseEntity.ok() .header("Surrogate-Key", // Fastly
    "product product-" + id + " category-" + p.getCategoryId()) .header("Cache-Tag", // Cloudflare
    "product product-" + id)
    .cacheControl(CacheControl.maxAge(Duration.ofMinutes(30)).cachePublic()) .body(p); } // On
product update – call CDN purge API: // Fastly: POST
https://api.fastly.com/service/{id}/purge/product-42 // Cloudflare: POST
/zones/{zone}/purge_cache {"tags":["product-42"]}
```

AWS CloudFront Configuration (CDK/Java)

```
// AWS CDK – CloudFront distribution for Spring Boot API CachePolicy apiCachePolicy =
CachePolicy.Builder.create(stack, "ApiPolicy") .cachePolicyName("ApiCachePolicy")
    .defaultTtl(Duration.minutes(5)) .maxTtl(Duration.hours(1)) .minTtl(Duration.seconds(0))
    .queryStringBehavior(CacheQueryStringBehavior.all())
    .headerBehavior(CacheHeaderBehavior.allowList( "Accept-Language", "Accept-Encoding"))
    .build(); Distribution dist = Distribution.Builder.create(stack, "ApiDist")
    .defaultBehavior(BehaviorOptions.builder() .origin(new HttpOrigin("api.example.com"))
    .viewerProtocolPolicy(ViewerProtocolPolicy.REDIRECT_TO_HTTPS) .cachePolicy(apiCachePolicy)
    .allowedMethods(AllowedMethods.ALLOW_ALL) .build()) .build(); // CloudFront invalidation (via
AWS SDK) CreateInvalidationRequest req = CreateInvalidationRequest.builder()
    .distributionId("E2QWRUHEXAMPLE") .invalidationBatch(InvalidationBatch.builder()
    .paths(Paths.builder() .items("/api/products/42", "/api/catalog/*") .quantity(2) .build())
```

```
.callerReference(UUID.randomUUID().toString()) .build()) .build();
cloudFrontClient.createInvalidation(req);
```

Cloudflare Cache Rules (API)

```
// Cloudflare Cache Rules (Page Rules replacement) // Configure via Cloudflare API or Terraform
// Example Page Rule: cache API for 5 minutes // URL: api.example.com/catalog/* // Cache Level:
Cache Everything // Edge Cache TTL: 5 minutes // Cloudflare purge by tag (Java SDK) HttpRequest
purgeRequest = HttpRequest.newBuilder()
.uri(URI.create("https://api.cloudflare.com/client/v4/zones/" + zoneId + "/purge_cache"))
.header("Authorization", "Bearer " + apiToken) .header("Content-Type", "application/json")
.POST(HttpRequest.BodyPublishers.ofString( "{\\\"tags\\\":[\\\"product-42\\\",\\\"category-5\\\"]}"))
.build(); HttpResponse<String> resp = httpClient.send(purgeRequest,
HttpResponse.BodyHandlers.ofString());
```

Origin Shield (Collapse of MISS Requests)

```
# Origin Shield topology: # Edge PoPs (200+) → Origin Shield (1 regional node) → Origin Server
# Without shield: 200 edge nodes each send 1 MISS request to origin # = 200 origin requests for
1 cache MISS event # With shield: All 200 edge nodes send MISS to shield # Shield collapses to
1 origin request # Shield caches the response, serves remaining edges # CloudFront: Enable
Origin Shield in origin settings # Fastly: Shielding PoP configuration # Cloudflare: Tiered
Caching (Smart Tiered Caching) – automatic
```

Fastly VCL Configuration

```
# Fastly VCL (Varnish Configuration Language) # vcl_recv: modify request before cache lookup
sub vcl_recv { # Strip cookies for public content (enables caching) if (req.url ~ "^/catalog/")
{ unset req.http.Cookie; } # Add cache bypass for authenticated requests if
(req.http.Authorization) { return(pass); } } # vcl_fetch: modify response from origin sub
vcl_fetch { # Override TTL for specific paths if (bereq.url ~ "^/static/") { set beresp.ttl =
365d; set beresp.http.Cache-Control = "public, max-age=31536000, immutable"; } # Remove
Set-Cookie to allow caching unset beresp.http.Set-Cookie; }
```

Cloudflare Workers (Edge Logic in JavaScript)

```
// Cloudflare Worker – runs at every PoP globally // 0ms cold start, V8 isolates, not
containers addEventListener('fetch', event => {
event.respondWith(handleRequest(event.request)); }); async function handleRequest(request) {
const url = new URL(request.url); // A/B test at edge – no origin needed if (url.pathname ===
"/") { const variant = Math.random() < 0.5 ? "A" : "B"; url.searchParams.set("variant",
variant); } // Cache API – programmatic cache control const cache = caches.default; let
response = await cache.match(request); if (!response) { response = await fetch(url.toString());
// Cache for 5 minutes const cloned = new Response(response.body, response);
cloned.headers.set("Cache-Control", "public, max-age=300"); event.waitUntil(cache.put(request,
cloned)); } return response; }
```

Interview Questions & Answers

Q1. What is a CDN and why is it used?

CDN (Content Delivery Network): globally distributed network of edge servers that cache content close to users. Reduces latency (200ms → 5ms), reduces origin server load (80-95% HIT rate for static content), improves availability during origin failures, and provides DDoS protection.

Q2. What is a CDN Point of Presence (PoP)?

A PoP is a CDN data center in a geographic region containing edge servers. Major CDNs have 100-300+ PoPs. Cloudflare: 300+ PoPs. CloudFront: 400+ PoPs. User requests are routed (via DNS Anycast or GeoDNS) to the nearest PoP, minimizing network hops.

Q3. How does a CDN decide to cache a response?

Based on HTTP headers from origin: Cache-Control: public + max-age/s-maxage instructs CDN to cache. Cache-Control: no-store or private prevents caching. Presence of Set-Cookie, Authorization request header, or query strings may bypass cache (configurable). s-maxage overrides max-age for shared caches.

Q4. What is an Origin Shield and why is it important?

Origin Shield is an additional CDN tier between edge PoPs and origin. When many PoPs miss simultaneously, they all route to shield (not origin). Shield deduplicates to 1 origin request. Critical for preventing origin stampede (thundering herd from CDN edge) and reducing origin bandwidth costs.

Q5. What is a Surrogate-Key / Cache-Tag?

Headers (Surrogate-Key in Fastly, Cache-Tag in Cloudflare) that tag a cached response with logical keys. On update, a single API call purges all cache entries with that tag across all PoPs. Example: tag product page with 'product-42 category-5'; on product update, purge 'product-42' tag.

Q6. How do you purge a CDN cache in real-time?

CloudFront: CreateInvalidation API (supports wildcards, ~1000/month free). Cloudflare: POST /zones/{zone}/purge_cache with URLs or Cache-Tags. Fastly: instant purge via Surrogate-Key (Fastly claims < 150ms global purge). Cost: CloudFront charges for > 1000 invalidations/month; Cloudflare and Fastly have tier limits.

Q7. What is a Lambda@Edge function?

AWS serverless functions (Node.js/Python) that run at CloudFront PoPs. Four trigger points: Viewer Request (before cache check), Origin Request (on MISS, before origin), Origin Response (after origin, before caching), Viewer Response (after cache, before client). Used for: authentication, URL rewriting, A/B testing, custom headers, geo-blocking.

Q8. What is a Cloudflare Worker?

JavaScript/WASM functions running at every Cloudflare PoP using V8 isolates. Zero cold start. Access to: fetch (HTTP requests), Cache API (programmable cache), KV (edge key-value store), Durable Objects (stateful edge). Used for: edge routing, authentication, request transformation, geo-based content, API gateway logic.

Q9. What is the difference between CloudFront and Cloudflare?

CloudFront: AWS service, tight AWS integration (S3, ALB, API Gateway, WAF), Lambda@Edge, pay-per-request pricing. Cloudflare: independent provider, DNS registrar + CDN + security, Workers (V8 isolates), 0ms cold start, free tier available, simpler DDoS protection, faster purge.

Q10. How does Fastly differ from other CDNs?

Fastly is a real-time CDN with instant purge (< 150ms globally via Surrogate-Key), VCL (Varnish Configuration Language) for deep cache logic customization, and Compute@Edge (WASM) for edge workers. Preferred for media/news sites requiring instant cache updates.

Q11. What HTTP headers should Java backends set for CDN optimization?

Cache-Control: public, max-age=60, s-maxage=300 (browser=1min, CDN=5min). Vary: Accept-Encoding (separate cache per encoding). Surrogate-Key for tag-based purge. ETag for conditional GET. Avoid Set-Cookie on cacheable responses. Remove unnecessary Vary headers.

Q12. What is the CDN cache warm-up problem?

After deployment or purge, all edge nodes have cold caches. First requests from each PoP miss to origin. If high traffic, origin gets hammered simultaneously. Solutions: Origin Shield (collapses misses), pre-warming scripts that hit key URLs at each PoP, staggered deployments, higher SWR window.

Q13. What is the Vary: Accept-Encoding gotcha in CDNs?

If Vary: Accept-Encoding is set, CDN caches separate entries for compressed and uncompressed responses. Without it, CDN may cache a gzip response and serve it to a client that doesn't support gzip. Most CDNs handle Accept-Encoding internally and strip it from the Vary consideration, but always test your CDN's behavior with compression headers.

Q14. How do you serve region-specific content via CDN?

CDNs add geo headers to origin requests: CloudFront-Viewer-Country (ISO country code), CF-IPCountry (Cloudflare), Fastly-Geo-Country. Origin uses these to vary response. Include Vary: CloudFront-Viewer-Country if different content per country — creates separate cache entries. Lambda@Edge or Workers can redirect/rewrite at edge without origin round-trip.

Q15. What is Cache-Control: stale-if-error and how does CDN use it?

Instructs CDN to serve stale content when origin returns 5xx or is unreachable, for the specified duration. stale-if-error=86400 serves stale for up to 24 hours on origin failure. Supported by Cloudflare, Fastly, CloudFront (via origin error caching). Provides high availability without complex fallback logic.

Q16. What is the difference between edge caching and browser caching?

Edge caching: CDN PoP; shared by all users; can be purged via CDN API; set by s-maxage or Cache-Control: public. Browser caching: private, per user, on device; cannot be purged externally; set by max-age with no public. Both use Cache-Control headers but require different directives for independent control.

Q17. What is an ESI (Edge Side Include)?

Edge Side Includes allow CDN to assemble a response from multiple cached fragments at the edge. HTML template with references. CDN fetches and caches each fragment independently with different TTLs. The dynamic personalized fragment can bypass cache while static chrome is served from edge. Supported by Varnish/Fastly, Akamai, Squid.

Q18. How do you handle API versioning in CDN caching?

URL versioning (/v1/, /v2/) naturally creates separate CDN cache entries. Include version in cache key. Avoid header-based versioning (Accept: application/vnd.v2+json) — requires Vary: Accept, which some CDNs handle poorly. URL versioning is CDN-friendly and explicit.

Q19. What is Anycast and how does CDN use it for routing?

Anycast: multiple CDN nodes share the same IP address. DNS resolves CDN domain to an anycast IP. Network routing (BGP) delivers the packet to the nearest topological PoP. Cloudflare and Fastly use Anycast for routing. CloudFront uses GeoDNS (DNS returns different A records per user region). Both achieve the goal of routing users to nearest PoP.

Q20. What are common CDN anti-patterns?

1) Caching per-user responses at CDN (use private). 2) Not setting s-maxage (CDN falls back to Expires or short defaults). 3) Cookie in responses preventing caching (strip session cookies from API responses). 4) Over-using wildcards in CloudFront invalidations (/ * = expensive, slow). 5) Not using Origin Shield (stampede on MISS events). 6) Same URL for different content variations without Vary header.

Q21. How does Cloudflare's Argo Smart Routing work?

Argo routes origin-bound requests over Cloudflare's private backbone (not public internet), selecting the fastest path based on real-time latency measurements across PoPs. Reduces origin fetch latency by 30-48% compared to public internet routing. Useful for dynamic, uncacheable requests that must hit origin.

Q22. What is the 'cookie poisoning' CDN problem?

If a response includes Set-Cookie and CDN caches it, all users receive the same cookie — a security vulnerability. Prevent: ensure cookies are never set on publicly cacheable responses, strip Set-Cookie at CDN (Fastly VCL: `unset beresp.http.Set-Cookie`), or use separate domains for authenticated API vs public CDN-cached content.

Q23. How do you test CDN cache behavior?

`curl -I https://cdn.example.com/api/products` — check X-Cache, Age, Cache-Control headers. Tools: hey (HTTP load tester), k6, WebPageTest (global HIT/MISS visibility). Monitor: CDN dashboard hit rate (should be > 80% for static, > 60% for API). Check Age header: increases over time for HIT; 0 on MISS.

Q24. What is CDN Request Collapsing?

When multiple requests arrive at a PoP for the same uncached resource simultaneously, CDN collapses them into a single origin request. All waiting requests receive the response once origin replies. Prevents thundering herd at CDN edge for popular content just after purge or on cold start. Supported by CloudFront, Fastly (request shielding), Varnish (grace mode).

Q25. Describe a complete CDN strategy for a Java microservices application.

1) Static assets: S3 + CloudFront, Cache-Control: `max-age=31536000 immutable, content-hashed` URLs. 2) Public API: `max-age=60, s-maxage=300, ETag, stale-while-revalidate=30, stale-if-error=3600`. 3) Personalized API: Cache-Control: `private, no-store` — bypass CDN. 4) Origin Shield: enable to collapse origin MISS traffic. 5) Surrogate-Key: tag responses; event-driven purge on data mutations via Kafka → Lambda. 6) CloudFront Functions: JWT validation, geo-blocking at edge.

Q26. What is a CDN configuration drift and how do you manage it?

CDN configuration drifts when engineers make manual changes to CDN settings that diverge from infrastructure-as-code (IaC). Manage via: Terraform `cloudflare/aws_cloudfront_distribution` resources, AWS CDK for CloudFront, Fastly's Terraform provider. All CDN changes go through Git + PR review. CDN drift causes inconsistent cache behavior across environments.

Q27. How does HTTP/2 Server Push interact with CDN caching?

HTTP/2 Server Push allows server to preemptively send resources to client. CDN intercepts pushes: CloudFront does not support Server Push. Modern recommendation: use Early Hints (103) instead — CDN can forward 103 Hint before fetching full response. Resource hints (`<link rel=preload>`) in HTML are more reliable than server push for CDN scenarios.

Q28. What are the differences between Cloudflare KV, Durable Objects, and Workers for caching?

KV (Key-Value): globally distributed, eventually consistent store; read latency <1ms globally; ideal for configuration and session data read-heavy workloads. Durable Objects: single-region strongly consistent objects for stateful edge computation (counters, rate limiters, real-time features). Workers Cache API: programmatic control of Cloudflare CDN cache — store/retrieve HTTP responses.

Q29. How do you implement geo-routing with CDN for a multi-region Java deployment?

Configure CDN with multiple origins per region. Use geo-based routing rules: EU users → EU origin, US users → US origin. Lambda@Edge or Cloudflare Workers can read `CloudFront-Viewer-Country` or `CF-IPCountry`, select origin URL, and forward request. Cache separately per region with Vary:

CloudFront-Viewer-Country or custom regional cache keys.

Q30. What is real-time purge vs TTL-based expiry and when to use each?

TTL expiry: simple, eventually consistent, no API cost, acceptable for long-lived content or data where slight staleness is OK (product catalog, blog posts). Real-time purge: immediate consistency, API call required on each content update, suitable for highly dynamic content (pricing, inventory) where stale data has business impact. Use both: short TTL as safety net + purge on known updates.